

Insidor

Architecture Reference

Version 4 prototype · March 2026

Production: <https://site-amber-one-38.vercel.app>

Frontend SPA

Privy Auth

Vercel Static

Simulated Data

Contents

1. Project overview
2. Tech stack
3. File structure
4. Views & navigation
5. Data models
6. Modal system
7. State management
8. UI components & sorting
9. Auth & wallet (Privy)
10. External integrations
11. Production vs prototype
12. Future backend considerations

1. Project overview

Insidor is a narrative-first Solana memecoin discovery and trading UI prototype. It surfaces viral social posts (X, TikTok, Reddit, Farcaster, Telegram), clusters them into **narratives**, links them to **tokens** (live or candidate), and provides simulated deploy/buy flows.

All market and social metrics are **mock/simulated** in the current build — the UI is designed to show the intended product experience before real APIs are wired in.

Entry points

Path	Role
site/index.html	Deploy copy — Vercel production static site (v4, full feature set)
Insidor-prototype (2).html	Older dev prototype (feed/research UX, Privy JS SDK core)
Desktop/insidorprototypev4.html	Local desktop copy kept in sync with deploy

Local development

```
npx serve . -p 3456
```

Serve from project root. Privy requires an `http://` origin (not `file://`). Add localhost to Privy Dashboard allowed origins.

Deploy

```
cd site
npx vercel deploy --prod --yes
```

Zero-build static deploy. No `vercel.json` — project linked via `site/.vercel/project.json`. Live URL aliased to **site-amber-one-38.vercel.app**.

2. Tech stack

Layer	Technology
Frontend	Vanilla HTML / CSS / JavaScript SPA (~2,560 lines in one file)
Fonts	Fontshare (Clash Display, General Sans), Google Fonts (JetBrains Mono)
Auth	Privy via CDN — <code>@privy-io/react-auth@1</code> + React 18 from <code>esm.sh</code>
Charts	TradingView widget (<code>tv.js</code>) on token page
Hosting	Vercel (static HTML, no build step)
Images	loremflickr.com (keyword-locked), fallback <code>picsum.photos</code>
Backend	None — all data in-memory in the browser

3. File structure

```
Insidor-app/
├─ site/
│  └─ index.html          ← production deploy app
│  └─ .vercel/project.json
├─ Insidor-prototype (2).html
├─ docs/                  ← this documentation
├─ package.json           ← @privy-io/js-sdk-core (prototype only)
└─ .cursor/rules/         ← project guardrails
```

site/index.html internal layout:

1. <head> — meta, Privy IDs, fonts, TradingView script, ~930 lines CSS
2. <body> — nav, .shell (main views + right rail), inline app script
3. Module script — Privy React mount into #userAuth

4. Views & navigation

Routing: VIEWS = ['trending', 'new', 'narratives', 'tokens', 'search', 'watchlist', 'token']

Functions: go(v), setView(v), viewStack, stepBack()

View ID	Nav	Purpose
v-trending	Yes	Dual top-10: narratives (24h viral gain) + tokens (trend score). Sub-tabs via #trendSubs
v-new	New Pairs	Newest tokens by age + pending candidates. No Hot/Gainers chips — newest first only
v-narratives	Yes	Full table of 14 narrative clusters. Sort chips: New, Trending. Column sort on Views/Gain/Mcap/Age
v-tokens	Yes	All live tokens + pending candidates. Chips: Hot / Gainers / New
v-watchlist	Yes	Session watchlists for narratives and tokens (star toggles)
v-search	No tab	Full-page search — reached via ⌘K → Enter or openSearchPage()
v-token	Hidden	Token detail page rendered into #tokenPage by openToken(sym)

Right rail (always visible on desktop)

460px sticky panel. Modes: **Trades**, **Launches**, **Viral** (railMode). Collapsible on token page via railCollapsed.

5. Data models

NARR[] — viral post feed (10 items)

Raw posts for rail, deploy source, trending cards.

```
{ plat, badge, lead, txt, views, posts, accts, peak, vel,
  who, at, t, pal, media } // media: photo|video|link|reply|none
```

Platforms: x, tt, rd, fc, tg. Tickers in text as \$TICKER.

TOKENS[] — live coins (10 seeded)

```
{ sym, name, ca, price, c5, c1h, c24, vol, liq, mc, ageM,
  lead, narrIdx, holders, live, txns,
  socials: { web, x, tg }, dex, pump,
  mintRevoked, freezeRevoked, lpBurned, top10, justDeployed? }
```

Created via `mkToken(sym, name, seed, narrIdx, lead)`.

PENDING{} — candidate tickers (no coin yet)

```
PENDING[sym] = { narrIdx, lead }
```

Shown in New Pairs, Launches rail, deploy availability checks. Starts empty in production site.

NARRATIVES[] — clusters (14 items)

```
{ id, title, blurb, imgSeed, image, createdAt, narrIdx,
  searchSeries, leadTimeMin, organicScore,
  posts: [ seedPost(...) ],
  tokens: [ seedTok(...) ] }
```

Example IDs: ferry-horn, norway-move, porch-frog, office-meme, etc.

Clusters without live tokens show a **Create** button instead of mcap.

seedPost() → post object

```
{ platform, handle, followers, text, image,
  views, replies, quotes, notable, sampleReplies[], postedAt }
```

seedTok() → narrative token metadata

```
{ ticker, name, mcap, liquidity, vol24h, holders, ageMin,
  firstDeployed, endorsedBy, canonical,
  safety: { mintRevoked, lpBurned, topHolderPct, ... },
  smartMoney: { smartWalletsIn, holderGrowth1h, topTraders[] } }
```

Key helper functions

- `getNarrative(id)`, `narrTopPost`, `narrTopToken`, `narrCombinedViews`
- `narrViralGain24h`, `narrLifecycle` → heating|peaking|cooling
- `extractTickers(txt)` — regex `\$[A-Z]{2,10}`
- `deriveTicker(txt)` — longest 3–10 letter word
- `syncNarTokens()` — sync narrative token mcap from live TOKENS

- `narrativeForToken(t)`, `tokenOriginStrip(t)` — token page → narrative link

6. Modal system

Single mount: `ensureModalRoot()` creates `#modalRoot` on `document.body`.

Shared close: `closeDeploy()` clears `innerHTML` + restores body scroll.

Modal	Backdrop ID	Open function
Deploy / Create coin	<code>#dback</code>	<code>openDeploy(idx)</code> , <code>openDeployForNarrative(id)</code>
Narrative detail	<code>#nmbback</code>	<code>openNarrative(id)</code>
Post detail	<code>#pdback</code>	<code>openPostDetail(...)</code>
⌘K search palette	<code>#palBack</code>	<code>openPalette()</code>
Deploy success	reuses deploy backdrop	<code>deployFiled()</code> replaces sheet content

Pattern: `.dback` (fixed fullscreen overlay) → `.sheet` (centered dialog). Body scroll locked while open. Escape closes palette first, then modal, then `stepBack()`.

7. State management

No `localStorage` / `sessionStorage` — all state is in-memory for the browser session.

Variable	Purpose
<code>viewStack</code>	Back navigation (starts ['trending'])
<code>trendSub</code>	<code>'narratives'</code> <code>'tokens'</code>
<code>railMode</code>	<code>'trades'</code> <code>'launches'</code> <code>'viral'</code>
<code>curSort</code> , <code>colSort</code> , <code>newColSort</code>	Token / new-pairs table sorting
<code>narrChipSort</code> , <code>narrSort</code>	Narrative table sorting
<code>searchQuery</code>	Shared search string (palette + search tab)
<code>WATCH_TOKENS</code> , <code>WATCH_NARR</code>	Watchlist Sets
<code>deployIdx</code> , <code>deployBuy</code> , <code>deployCluster</code>	Deploy modal state
<code>chartToken</code> , <code>curSide</code> , <code>slip</code> , <code>prio</code>	Token page trade panel
<code>tvWidget</code>	TradingView widget instance

Live updates via `setInterval`: rail stream (2.6s), SOL price (2s), token prices (1.5s), trade feed (2s on token page).

8. UI components & sorting

Layout

- `.shell` — CSS grid: main + 460px rail
- `.view.on` — active main section
- `.tttable` + `.thd` / `.nthd` — data tables

Sort systems

- **Narratives:** chips New | Trending; columns views, gain, mcap, age
- **Tokens:** chips Hot (vol) | Gainers (c24) | New (ageM); column override via header click
- **New pairs:** always newest first (ageM); column sort optional
- **trendScore(t):** weighted vol/mc + txns + c24 + c1h

Visual components

- `sparkSVG(t)` — 24h mini chart in token rows
- `narrSparkSVG` — narrative search trend sparkline
- `starTok` / `starNarr` — watchlist toggles
- `.pbadge` — platform badges (X, TikTok, etc.)
- `.tp-origin` — token page source narrative strip

9. Auth & wallet (Privy)

Meta tags: `privy-app-id`, `privy-client-id` in HTML head.

React module mounts `PrivyProvider` into `#userAuth`:

- Unauthenticated → **Connect** button → `login()`
- Authenticated → user chip (Google name / email / wallet) + logout
- Fallback if file:// or SDK load failure

Trade execution is mock — `#exec` simulates confirm after 700ms. Copy references user signing in their own wallet.

10. External integrations

Service	Usage
TradingView	<code>TradingView.widget</code> — maps tokens to BINANCE memecoin proxy symbols
Privy	OAuth + wallet connect; origin allowlist in dashboard
loremflickr / picsum	Meme thumbnails via <code>imgURL(kw, seed, size)</code>
DexScreener	<code>https://dexscreener.com/solana/{ca}</code>
pump.fun	<code>https://pump.fun/{ca}</code>
Social platforms	Simulated permalinks via <code>postURL(p)</code>

Planned real data sources (commented in code): X/TikTok API, Google Trends, DexScreener/Birdeye, RugCheck/GMGN, Helius wallet labels.

11. Production vs prototype

Aspect	site/index.html	Insidor-prototype (2).html
UX	Multi-tab terminal (Trending, Narratives, Tokens, etc.)	Feed + Research terminal
Auth	Privy React SDK	Privy JS SDK core + Solana SIWS
Charts	TradingView widget	Older canvas approach
Narratives	Full NARRATIVES cluster system	Simpler state object
Deploy	Vercel static	Local prototype

12. Future backend considerations

For a production architecture, expect to add:

1. **Narrative ingestion pipeline** — scrape/API feeds from X, TikTok, Reddit; cluster into narratives
2. **Token index** — DexScreener/Birdeye/Helius for live prices, liquidity, safety scores
3. **Deploy service** — pump.fun or custom mint; wallet signing via Privy embedded wallet or external
4. **Trade execution** — Jupiter or Raydium swap API; slippage/priority fee handling
5. **User persistence** — watchlists, preferences, deploy history (DB + auth user ID from Privy)
6. **WebSocket feeds** — live trades rail, price ticks, new pair launches